Miniprojekt: Studyplanner

Lernziele . . . 1
Bewertung . . . 1
Ausgangslage . . . 2
Vorlage . . . 3
Teil 1: Objektorientiertes Design / Modellierung (Kursblock 4) . . . 7
Teil 2: Implementationsaufgaben . . . 9
Kursblock 5: Collections . . . 9
Kursblock 6: Packages und Exceptions . . . 10
Kursblock 7: Input / Output (IO) . . . 11
Kursblock 8: Lambdas und das Stream-API . . . 12
Kursblock 9: Records . . . 13
Kursblock 10: kein neues Thema . . . 13
Git Repositories . . . 13
Tipps & Hinweise zur Abgabe . . . 14

Lernziele

Das Ziel dieses Miniprojekts ist, die gelernten Konzepte der OOP-Lerneinheit in einem konkreten Projekt zu nutzen. Insbesondere sollen folgende Konzepte angewendet und vertieft werden:

- Java-Grundlagen (Datentypen, Variablen, Ausdrücke, Kontrollstrukturen, Blöcke und Methoden)
- Klassen, Objekte und Referenzen
- Objektorientiertes Design mit UML Klassen- und Sequenzdiagrammen (OOAD)
- Collections wie Listen, Sets und Maps
- Packages
- Input/Output (IO)
- Lambdas und das Stream-API
- Records
- Exceptions

Bewertung

Bezüglich der Bewertung dieses Leistungsnachweises (Bewertungsraster) verweisen wir auf den entsprechenden Detailauftrag.

Ausgangslage

Sie sollen eine Anwendung entwickeln, mit welcher Studierende unterschiedlicher Studiengänge sich einen Studienplan erstellen können. Aufgrund der Abhängigkeiten zwischen Modulen, soll das Programm berechnen, in welchen Semestern welche Module besucht werden können. Und damit auch wie viele Semester es für das Studium benötigt.

Betrachten wir ein Beispiel. Ein Student möchte Informatik an der OST studieren und dabei folgende Module belegen:

Modul	Voraussetzungen
DB1	OO
DB2	DB1
Math	-
OO	-
AD1	OO
CPI	OO Math
Thesis	DB2 SE2 UI2
SE1	AD1 CPI DB1
SE2	DB1 SE1 UI1
UI1	AD1
UI2	UI1

Aufgrund der Voraussetzungen gewisser Module, gäbe es die Möglichkeit, dieses nach folgendem Plan zu besuchen:

Semester 1	Semester 2	Semester 3	Semester 4	Semester 5
Math	DB1	DB2	SE2	Thesis
OO	AD1	SE1	UI2	
	CPI	UI1		

Zur Einfachheit wird angenommen, dass jedes Semester alle Module stattfinden und beliebig viele Module pro Semester belegt werden dürfen. Sie müssen also keine weiteren Constraints berücksichti-

gen und dürfen auch davon ausgehen, dass es keine Beschränkung der Studiendauer gibt. Sie dürfen so viele Semester verplanen wie aufgrund der Input-Daten notwendig sind.

Vorlage

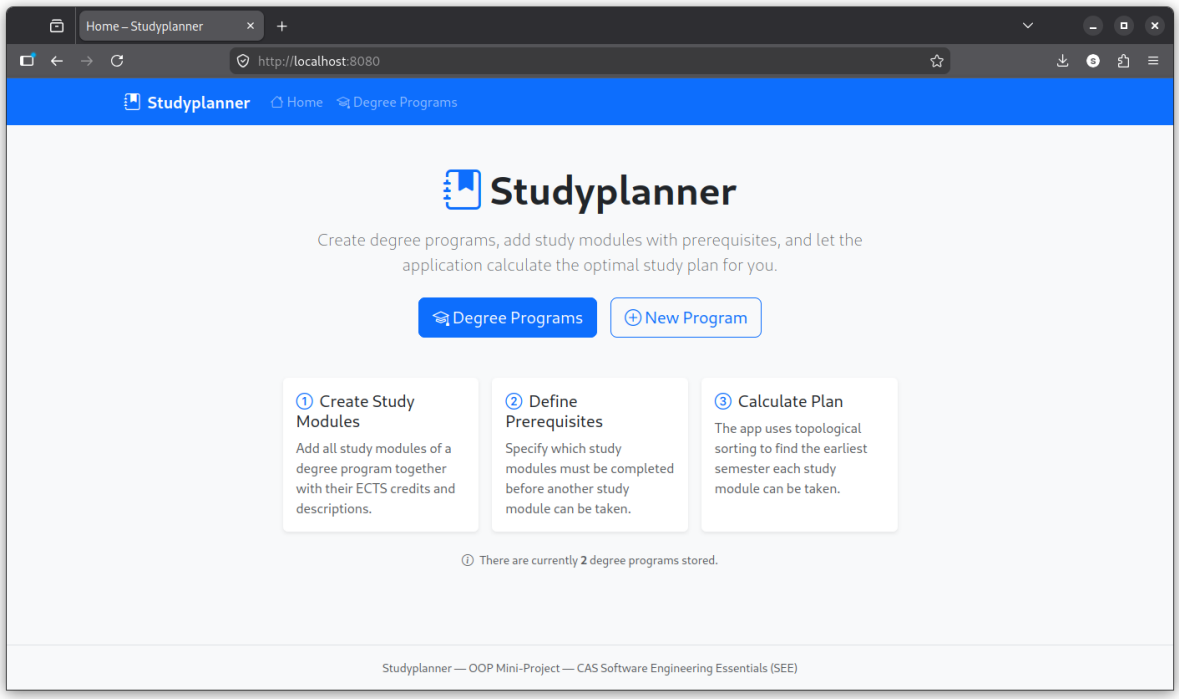
Als Vorlage erhalten Sie eine Spring Boot Anwendung mit einem fertigen User Interface (UI). Sie müssen in diesem Projekt also lediglich die Logik des Programms in diese Vorlage einbauen; es ist grundsätzlich keine UI-Entwicklung notwendig.

Das UI unterstützt das Erstellen beliebiger Studiengänge (“Degree programs”). Zu jedem Studiengang können dann Module (“Modules”) und deren Voraussetzungen (“Prerequisites”) erstellt werden. Zum Schluss kann ein Studienplan (“Schedule”) generiert werden.

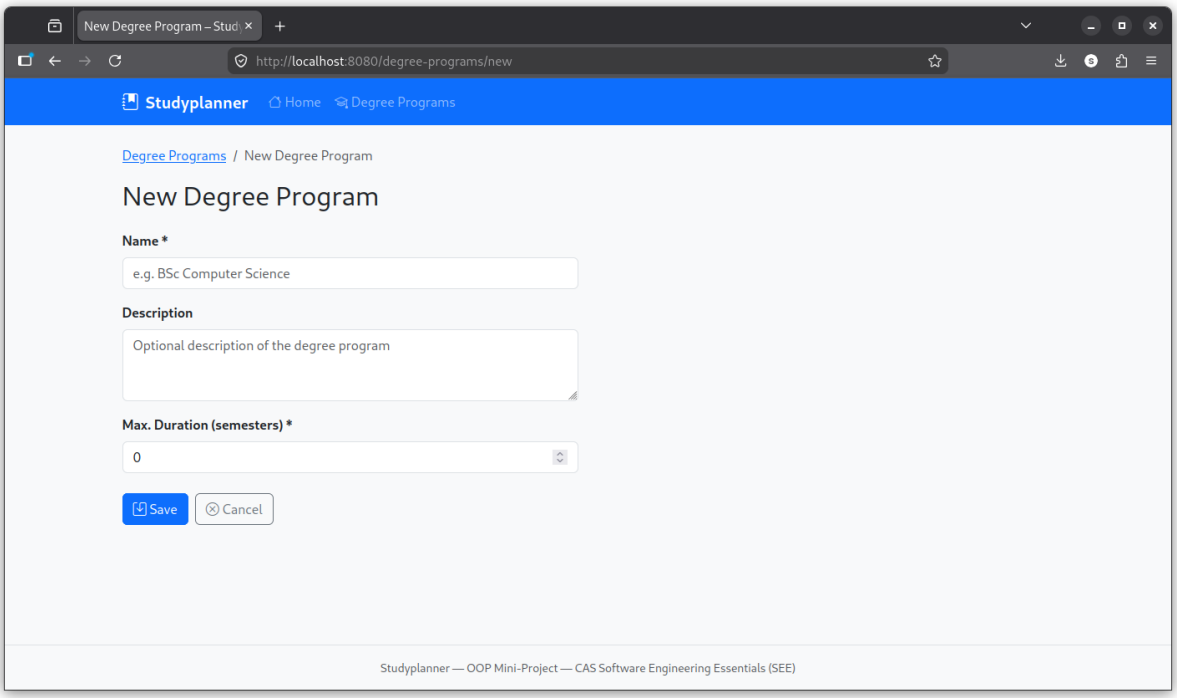
Beim Kickoff im Kursblock 4 (22. Mai 2026) zeigen wir Ihnen eine Demo, wie die fertige Applikation funktionieren soll.

Folgende Screenshots zeigen die einzelnen Anwendungsfälle im UI.

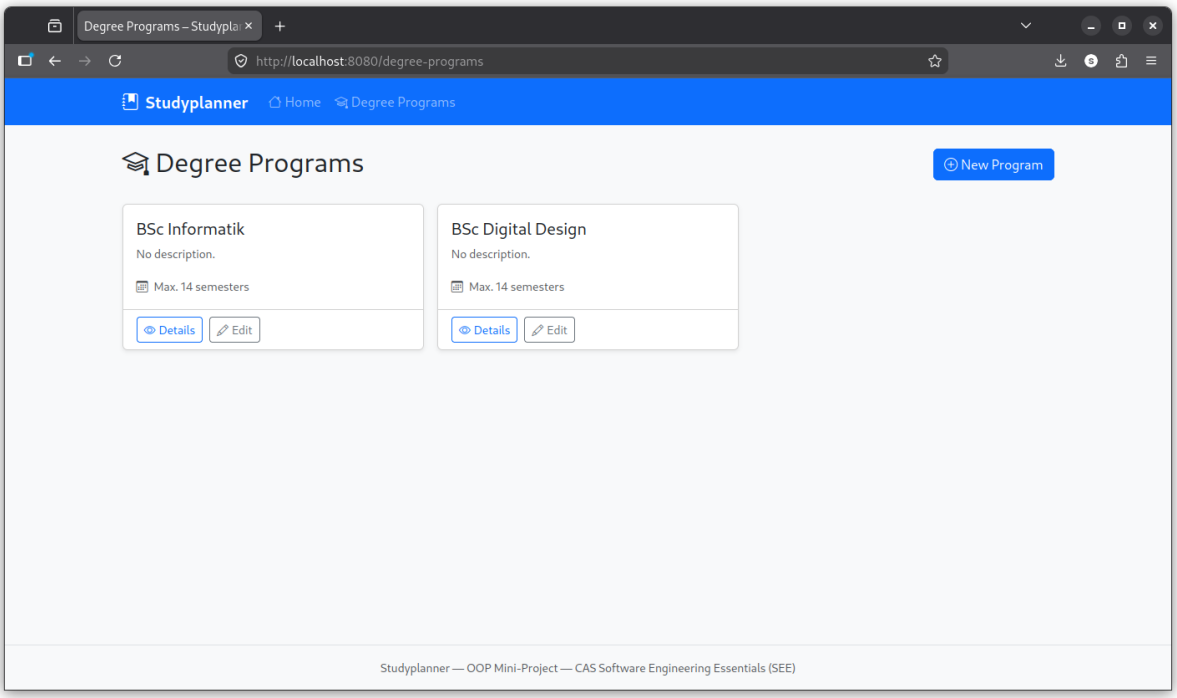
Der Start-Screen sieht wie folgt aus:



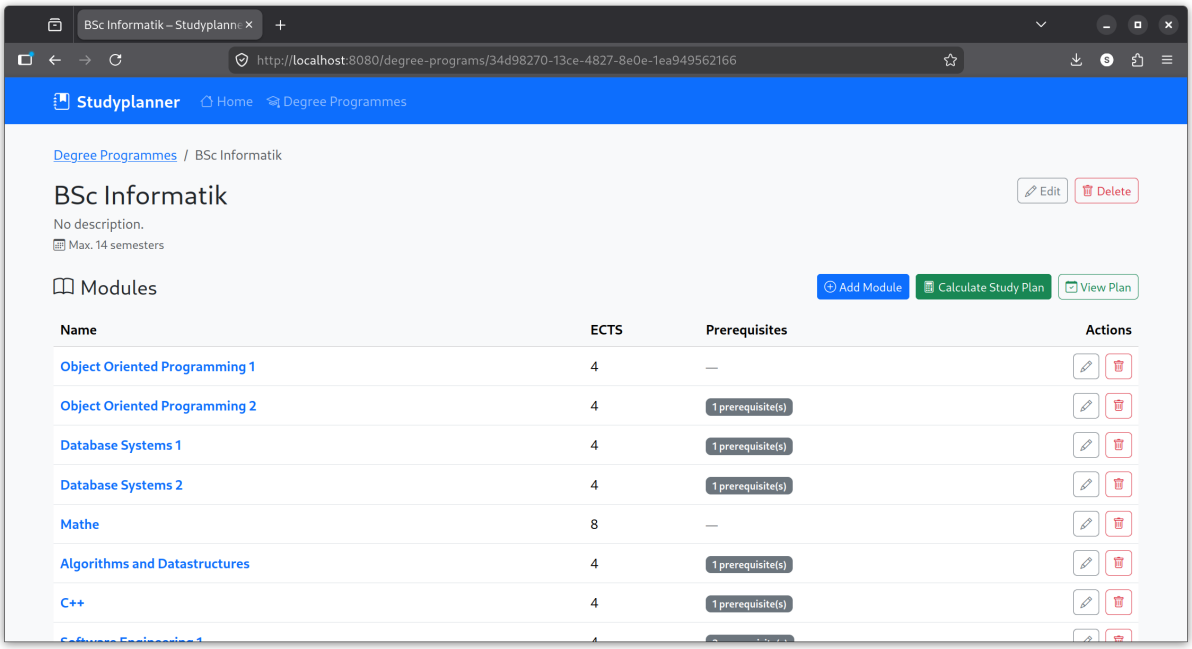
Über “New Program” kann ein neues Studienprogramm erstellt werden:



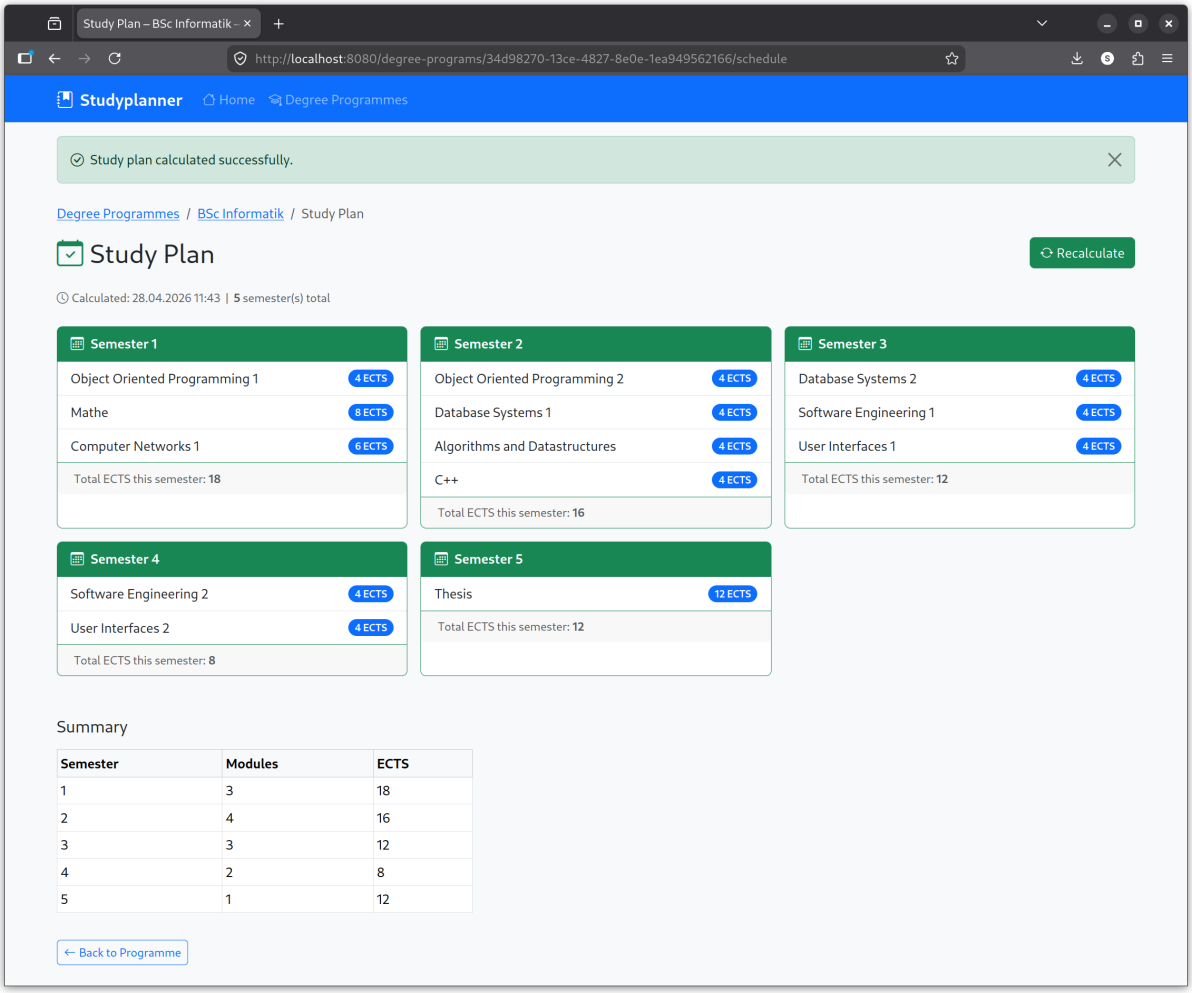
Über “Degree Programs” kann man eine Liste aller Studiengänge anzeigen:



Hat man einmal einen Studiengang, kann man auf der Detailseite die Module verwalten (erstellen, bearbeiten, löschen):


BSc Informatik – Studyplanner
http://localhost:8080/degree-programs/34d98270-13ce-4827-8e0e-1ea949562166
Studyplanner
Home
Degree Programmes
Degree Programmes / BSc Informatik
BSc Informatik
No description.
Max. 14 semesters
Edit
Delete
Modules
Add Module
Calculate Study Plan
View Plan
Name
ECTS
Prerequisites
Actions
Object Oriented Programming 1 4 —
Object Oriented Programming 2 4 1 prerequisite(s)
Database Systems 1 4 1 prerequisite(s)
Database Systems 2 4 1 prerequisite(s)
Mathe 8 —
Algorithms and Datastructures 4 1 prerequisite(s)
C++ 4 1 prerequisite(s)

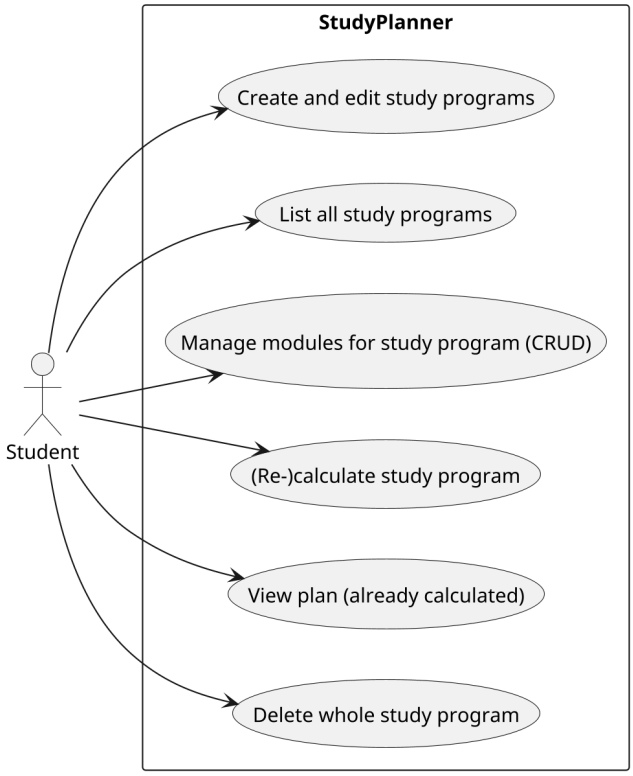
Über den Button “Calculate Study Plan” wird dann der Studienplan berechnet und wie folgt angezeigt:



Teil 1: Objektorientiertes Design / Modellierung (Kursblock 4)

Im ersten Teil am 22. Mai (Kursblock 04) geht es darum, dass Sie die Problemstellung erstmal modellieren mittels Klassen- und Sequenzdiagrammen. Die wichtigsten Konzepte der Domäne haben wir oben, unter anderen auf dem UI, bereits gesehen.

Wir haben als erstes Artefakt der objektorientierten Analyse bereits ein Use Case Diagramm erstellt, welches die wichtigsten Anwendungsfälle aufzeigt:



Aufgabe 0: Importieren Sie die Ausgangslage ins IntelliJ und machen Sie sich mit dem bereits vorhandenen Code vertraut.

Aufgabe 1: Entwickeln Sie in der Gruppe ein Design in Form eines Klassendiagramms. Das Modell soll die obigen Use Cases abdecken. Um technische Details, z.B. wie Sie Daten persistieren, müssen Sie sich in diesem Schritt noch nicht kümmern.

Wir benötigen folgende Informationen in unserem System, welche der Benutzer eingeben muss:

- **Studiengang** (degree program): Name, Beschreibung, maximale Anzahl Semester (muss in der Berechnung des Studienplans nicht berücksichtigt werden)
- **Modul** (study module): Eindeutige Abkürzung (Key; z.B. “OOP1” für “Objektorientierte Programmierung 1”), Name, Beschreibung, Anzahl ECTS

  - Ab Kursblock 8: “Angeboten von” (offered from), “Angeboten bis” (offered to); jeweils als *LocalDate*

Folgende Daten braucht der Studienplan, der dann generiert werden soll:

- **Studienplan** (schedule): Liste von Semestern (jedes Semester besteht wiederum aus einer Liste von Modulen)

Tipp: Denken Sie neben der Modellierung der Domänenobjekte auch an Verantwortlichkeiten innerhalb der Applikation und wie Sie diese sauber trennen. Zum Beispiel braucht es vielleicht eine Komponente, welche die Studienplanung macht. Ausserdem müssen die Daten irgendwo verwaltet werden (erstmal in Collections, später müssen sie dann persistiert werden). Schauen Sie, dass Sie all diese Logik nicht direkt in die Controller integrieren. Weshalb? Überall wo dort jetzt ein TODO steht, soll idealerweise nur ein Methodenaufruf einer anderen Komponente eingefügt werden und nicht viel Logik/Code.

Aufgabe 2: Erstellen Sie für den Anwendungsfall der Berechnung des Studienplans ein Sequenzdiagramm. Die restlichen CRUD-Anwendungsfälle sind alle sehr ähnlich. Erstellen Sie für ein oder maximal zwei weitere Anwendungsfälle noch ein Sequenzdiagramm.

Tipp: Kommen Sie nach diesem Schritt in einer Übungslektion auf uns zu und holen Sie Feedback zu Ihren Modellen.

Teil 2: Implementationsaufgaben

In den folgenden Kursblöcken 5 - 10 werden Sie die Anwendung Schritt für Schritt ausbauen. Dabei nutzen Sie die Konzepte der jeweiligen Vorlesungen in Kursblöcken. Nachfolgend finden Sie die groben Aufgabenstellungen je Kursblock.

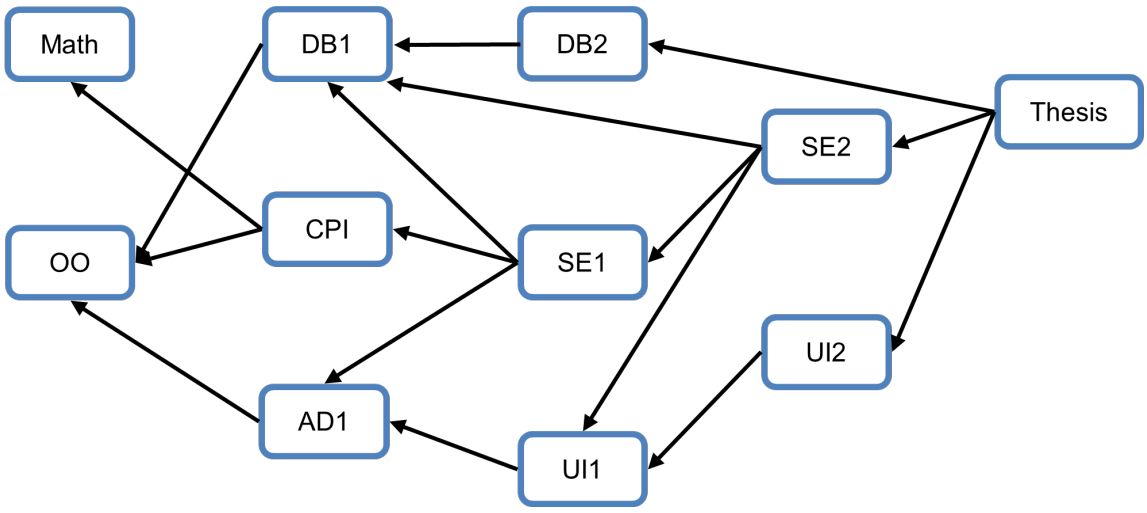
Disclaimer: Details der Aufgabenstellungen können sich im Verlauf der Lerneinheit noch ändern.

Kursblock 5: Collections

Aufgabe 1: Implementieren Sie mittels der gelernten Collections eine erste Version der Berechnung des Studienplans. Sie dürfen die Module dabei auch noch "hard-coded" in einer Methode erzeugen. Sie müssen noch nicht zwingend übers UI erstellbar sein.

Tipp zur Lösung; implementieren Sie den Algorithmus wie folgt:

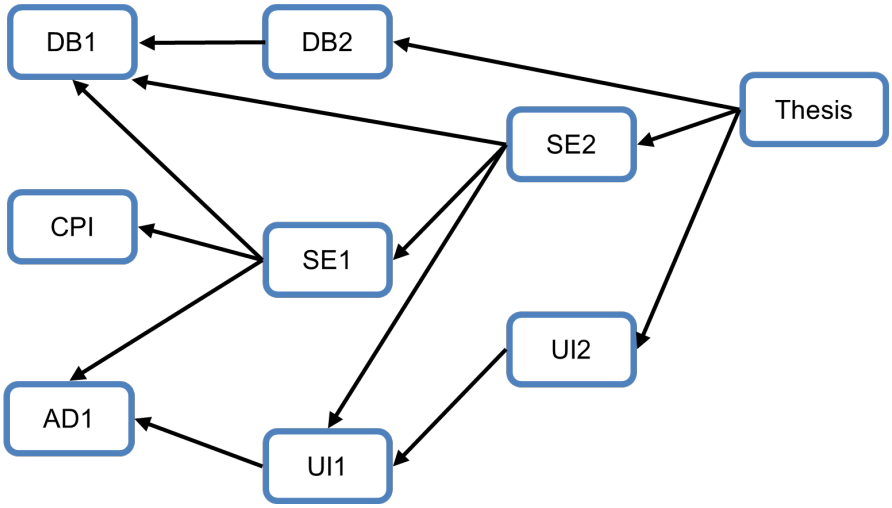
1. Erstellen Sie für jedes Modul genau ein Objekt. Das Objekt enthält eine Collection seiner vorausgesetzten Module bzw. der Namen seiner vorausgesetzten Module (letzteres ist einfacher). Beispiel der Objekt-Struktur:



2. Suchen Sie die Module mit leerer Voraussetzungs-Collection. Diese können im aktuellen Semester besucht werden. Beispiel: Am Anfang haben "Math" und "OO" keine Voraussetzungen und können also belegt werden.
3. Speichern Sie die im aktuellen Semester besuchten Module ab bzw. geben Sie diese aus:

```
1 Semester 1: Math OO
```

4. Entfernen Sie nun die besuchten Module als Voraussetzung aus den restlichen Modulen. Beispiel: “Math” und “OO” werden nun aus der Struktur entfernt. Danach sieht die Struktur so aus:



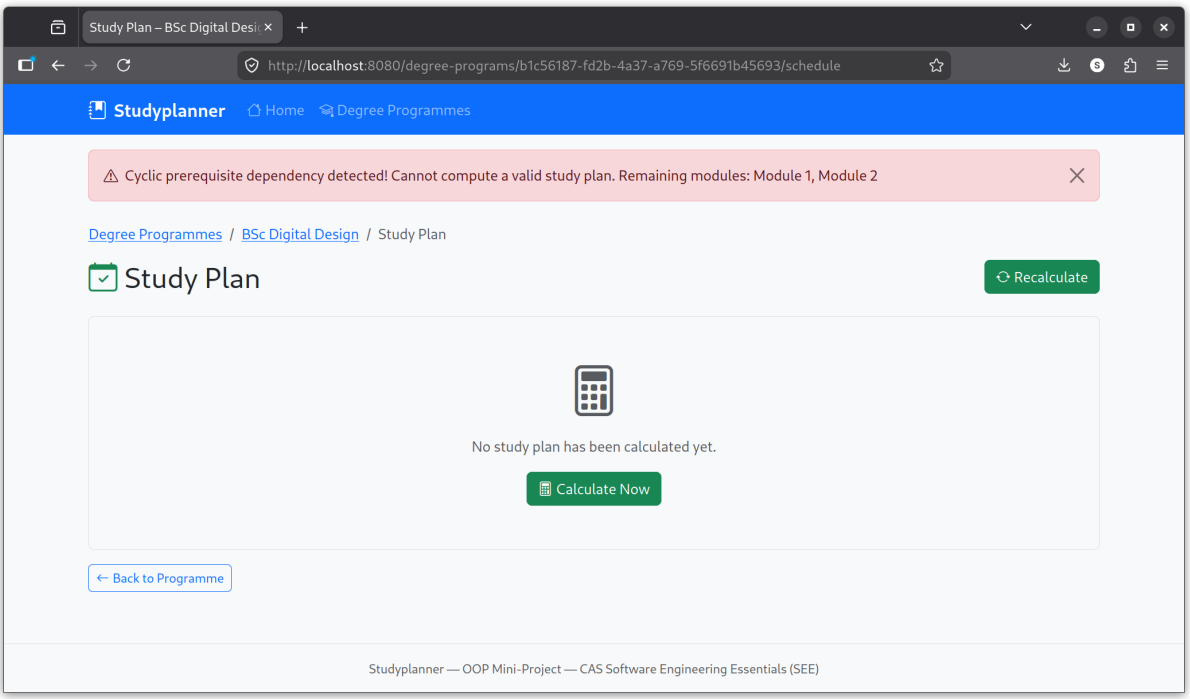
5. Gehen Sie zum nächsten Semester und wiederholen Sie den Schritt 2, solange es noch Module gibt.

Aufgabe 2 (potentiell ongoing über mehrere Wochen): Implementieren Sie die CRUD-Anwendungsfälle (Create, Read, Update, Delete) für die Studiengänge und Module. Sie können die Daten einfach in Collections speichern; sie müssen noch nicht über den Neustart der Applikation hinaus persistiert werden. Damit initial nach dem Start der Applikation Daten vorhanden sind, können Sie diese einfach “hard-coded” erzeugen und in die Collections einfügen. Als weiteren Schritt integrieren Sie die Generierung des Studienplans mit dem UI und zeigen den generierten Plan dort an. Hinweis: Die Spalte “Module Keys” in der Summary-Table dürfen Sie noch leer lassen. Dies implementieren wir in Kursblock 8 mit der Stream API.

Kursblock 6: Packages und Exceptions

Aufgabe 1: In der Vorlesung haben Sie nun Packages im Detail kennengelernt. Falls Sie dies nicht schon umgesetzt haben, überlegen Sie in der Gruppe, was eine sinnvolle Package-Struktur für Ihr Projekt sein könnte, und setzen Sie diese um.

Aufgabe 2: Wir möchten nun noch eine Fehlerbehandlung mit Exceptions ergänzen. Wenn die Eingabedaten einen Zyklus enthalten, also z.B. Module gegenseitig abhängig sind, soll dies erkannt und ein Fehler ausgegeben werden:



Wenn in Schritt 2 (siehe oben) dieser speziellen Abhängigkeiten beachtenden Sortierung (Fachbegriff topologische Sortierung) alle Module Voraussetzungen haben, sind die Abhängigkeiten zirkulär und es gibt keinen möglichen Studienplan. Erkennen Sie solche zyklischen Abhängigkeiten in Ihrem Programm und geben Sie eine Fehlermeldung aus. Entwickeln Sie dazu eine `CyclicDependencyException`, welche in diesem Fall ausgelöst wird.

Passen Sie den `ScheduleController` so an, dass die Exception dort gefangen wird und die `message` auf dem UI angezeigt wird. Sie finden dazu einen `TODO` im Code.

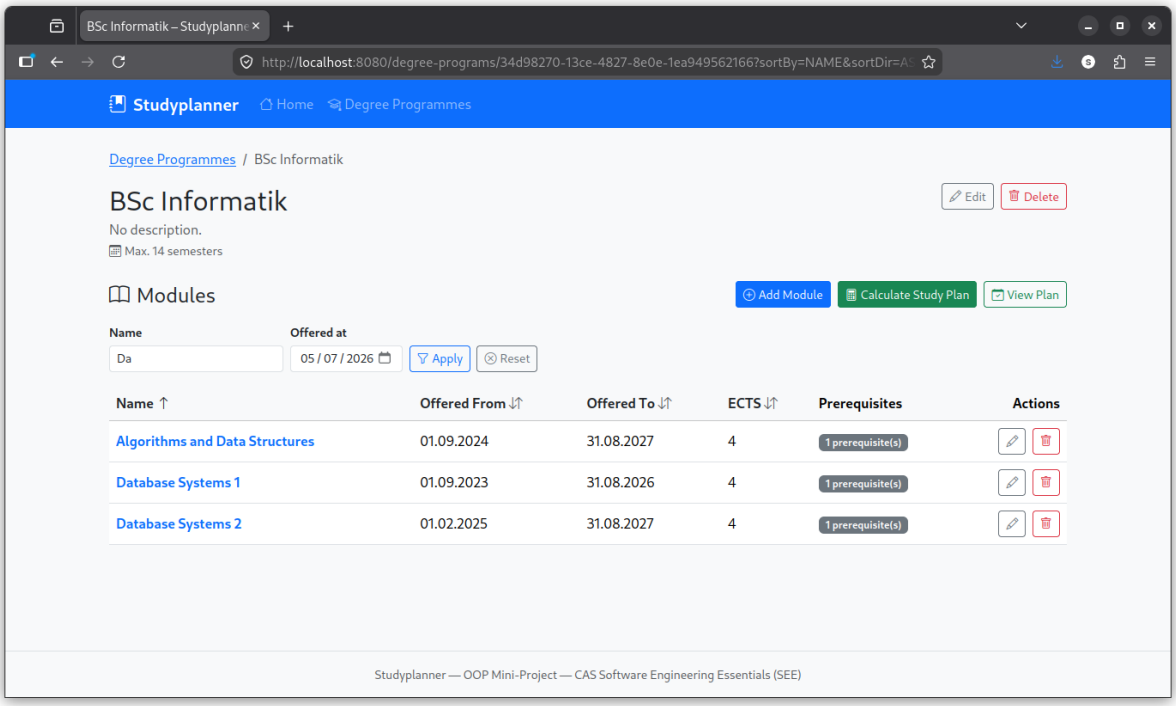
Kursblock 7: Input / Output (IO)

Aufgabe: In diesem Kursblock geht es nun darum, die Daten die der Benutzer eingibt (Studiengänge, Module) und auch generierte Studienpläne zu persistieren. Die Daten sollen in einer JSON-Datei abgespeichert werden. Wenn man die Spring Boot Anwendung stoppt und neu startet, sollen die Daten wieder geladen werden. Nutzen Sie Jackson (wie in der Vorlesung gelernt) um die Daten zu serialisieren und zu deserialisieren.

Tipp: Erstellen Sie pro Entität (DegreeProgram, StudyModule, Schedule) eine Klasse, welche sich um die Persistenz kümmert und Methoden für die CRUD-Operationen anbietet. Solche Klassen werden auch Repositories genannt. Vorschlag: Erstellen Sie folgende Klassen welche bei jeder Lese-Operation aus dem entsprechenden JSON-File lesen und bei jeder Schreib-Operation das JSON neu speichern: *DegreeProgrammRepository*, *StudyModuleRepository*, *ScheduleRepository*

Kursblock 8: Lambdas und das Stream-API

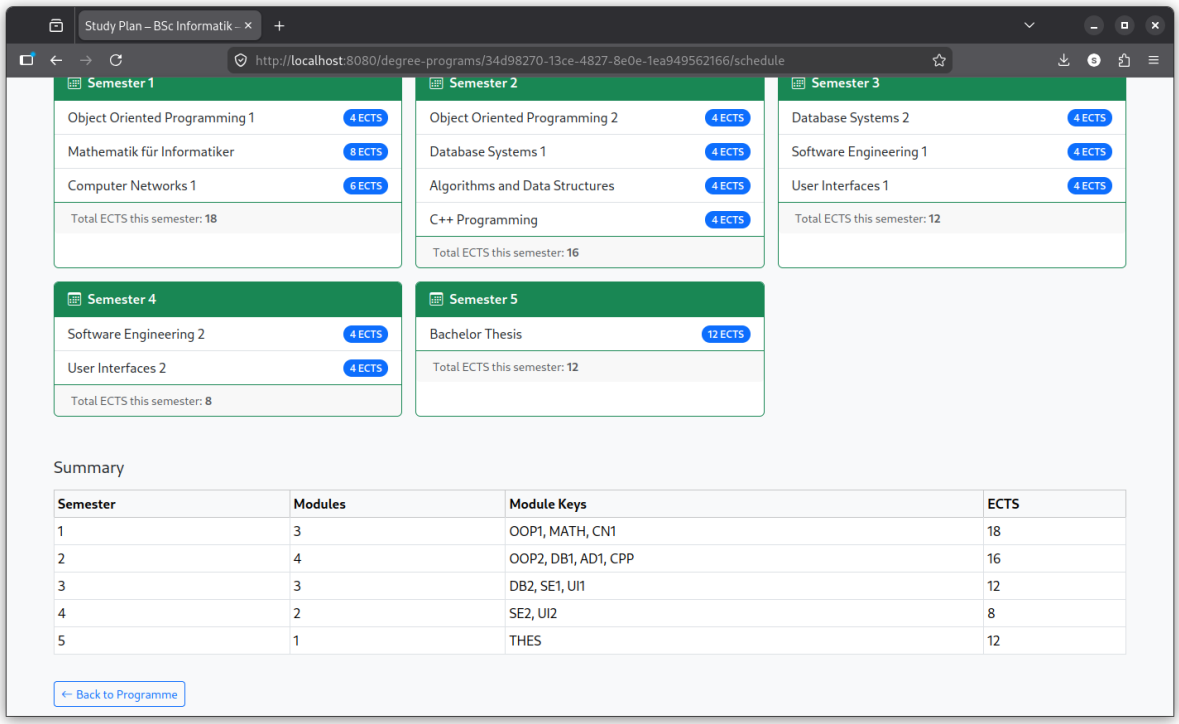
Aufgabe: In der Detailansicht des Studiengangs bzw. der Liste der Module wollen wir nun noch Funktionalitäten zum Filtern und Sortieren einbauen:



Implementieren Sie folgende **Sortier- und Filter-Features**:

- Filtern nach Name (siehe Screenshot/UI)
- Filtern nach Modulen die zu einem gewissen Datum gültig sind (“Offered at”; default: heute)
  - Dazu müssen Sie die Modul-Klasse erweitern, falls Sie die Informationen noch nicht im Modell haben: Ein Modul soll nur über einen Zeitraum angeboten werden können. Dafür braucht es zwei Datums-Felder, “offeredFrom” und “offeredTo”. Benutzen Sie den Typ *LocalDate*.
- Sortieren nach *Name*, *Offered From*, *Offered To*, *ECTS*
  - Jeweils Aufsteigend (ASC) oder Absteigend (DESC) möglich (im UI mit den Pfeilen bereits umgesetzt).

Zusätzlich möchten wir noch die **Summary-Table** aktualisieren bzw. jetzt die Spalte “Module Keys” noch wie folgt umsetzen:



Kursblock 9: Records

Aufgabe: Überlegen Sie, wo es in Ihrem System Klassen gibt, welche Sie zu Records umbauen könnten. Führen Sie dann die entsprechenden Refactorings durch.

Hinweis: Es gibt bereits in der Ausgangslage gewisse Klassen, wo man stattdessen Records nutzen könnte.

Kursblock 10: kein neues Thema

In der letzten Veranstaltung am 03. Juli 2026 gibt es keine neuen Themen mehr. In diesen Lektionen können Sie am Projekt arbeiten und sind entsprechend betreut. Stellen Sie die letzten wichtigen Fragen und klären Sie Unklarheiten vor der **Abgabe am 10. Juli 2026**.

Git Repositories

Unter gitlab.ost.ch/mas-se/oop/projects-2026 finden Sie das Git-Repository für Ihre Gruppe. Wir empfehlen Ihnen hier zu arbeiten, es ist aber keine Pflicht. Sie können sich auch anders organisieren. Falls Sie ein anderes Git-Repo/System benutzen wollen, geben Sie uns Zugriff darauf. Warum? **Wir**

möchten sehen, dass alle Teammitglieder am Projekt mitarbeiten. Stellen Sie sicher, dass wir dies am Ende des Projekts der Git-History entnehmen können.

Die URL zum jeweiligen Git-Repo jeder Gruppe ist wie folgt: `https://gitlab.ost.ch/mas-se/oop/projects-2026/studyplanner-2026-grp{group-number}` (jede Gruppe hat nur Zugriff auf das eigene Repo)

Tipps & Hinweise zur Abgabe

Generelle Tipps:

- Sie werden die zu Beginn erstellten Klassen- und Sequenzdiagramme im Verlauf des Projekts wieder anpassen müssen. Das erste Design ist nicht in Stein gemeisselt. Schauen Sie, dass Sie am Ende Diagramme abgeben, welche mit dem Code konsistent sind.
- Schreiben Sie, wo immer möglich, auch Unit Tests.
- Sie müssen vielleicht mit unbekannter Technologie arbeiten (Spring, JUnit Tests, Git, Maven, etc.). Wir sind uns dessen bewusst. Wenn Ihnen Dinge unklar sind, melden Sie sich bei uns und stellen Sie Fragen.
- Hilfreiche Links zum Kennenlernen/Experimentieren von Spring: (keine Pflicht)
  - Spring Boot
  - Spring Initializr, falls Sie zum Experimentieren mal ein eigenes/neues Spring Projekt generieren möchten.

Beachten Sie folgend Hinweise zur **Abgabe** Ihres Projekts:

- Geben Sie das Projekt als ZIP-Datei via Moodle ab. Zur Abgabe geht es hier: Abgabe Mini-Projekt (Studyplanner)
  - Ihre Gruppen sind in Moodle abgebildet. Nur eine Person der Gruppe muss die ZIP-Datei hochladen.
  - Deadline: **Freitag, 10. Juli 2026, 23:59**
- Denken Sie daran, nicht nur den Code, sondern auch die Modelle/Diagramme abzugeben.
- Geben Sie zu allen Diagrammen (Klassen- und Sequenzdiagramme) nicht nur gerenderte Bilder, sondern auch die Sourcen (PlantUML oder draw.io) ab.
- Schauen Sie vor der Abgabe darauf, dass Sie den Code bereinigen (keine unnötigen Code-Überbleibsel, etc.).
- Schauen Sie sich den Bewertungsraster des Detailauftrages an und kommen Sie auf uns zu, falls es Fragen dazu gibt.